maix.fs

maix.fs module

> You can use `maix.fs` to access this module with MaixPy
> This module is generated from MaixPy and MaixCDK

1. Module

No module

2. Enum

2.1. SEEK

SEEK enums

item	describe
values	**SEEK_SET**: Seek from beginning of file. **SEEK_CUR**: Seek from current position. **SEEK_END**: Seek from end of file.

> C++ defination code:

```
enum SEEK
    {
        SEEK_SET = 0,  // Seek from beginning of file.
        SEEK_CUR = 1,  // Seek from current position.
        SEEK_END = 2,  // Seek from end of file.
    }
```

3. Variable

4. Function

4.1. isabs

```
def isabs(path: str) -> bool
```

Check if the path is absolute path

item	description
param	**path**: path to check
return	true if path is absolute path

> C++ defination code:
>
> ```
> bool isabs(const std::string &path)
> ```

4.2. isdir

```
def isdir(path: str) -> bool
```

Check if the path is a directory, if not exist, throw exception

item	description
param	**path**: path to check
return	true if path is a directory

> C++ defination code:
>
> ```
> bool isdir(const std::string &path)
> ```

4.3. isfile

```
def isfile(path: str) -> bool
```

Check if the path is a file, if not exist, throw exception

item	description
param	**path**: path to check
return	true if path is a file

> C++ defination code:
>
> ```
> bool isfile(const std::string &path)
> ```

4.4. islink

```
def islink(path: str) -> bool
```

Check if the path is a link, if not exist, throw exception

item	description
param	**path**: path to check
return	true if path is a link

> C++ defination code:
>
> ```
> bool islink(const std::string &path)
> ```

4.5. symlink

```
def symlink(src: str, link: str, force: bool = False) -> maix.err.Err
```

Create soft link

item	description
param	**src**: real file path **link**: link file path **force**: force link, if already have link file, will delet it first then create.

> C++ defination code:
>
> ```
> err::Err symlink(const std::string &src, const std::string &link, bool force = false)
> ```

4.6. exists

```
def exists(path: str) -> bool
```

Check if the path exists

item	description
param	**path**: path to check
return	true if path exists

> C++ defination code:
>
> ```
> bool exists(const std::string &path)
> ```

4.7. mkdir

```
def mkdir(path: str, exist_ok: bool = True, recursive: bool = True) -> maix.err.Err
```

Create a directory recursively

item	description
param	**path**: path to create **exist_ok**: if true, also return true if directory already exists **recursive**: if true, create directory recursively, otherwise, only create one directory, default is true
return	err::ERR_NONE(err.Err.ERR_NONE in MaixPy) if success, other error code if

item	description
	failed

C++ defination code:

```
err::Err mkdir(const std::string &path, bool exist_ok = true, bool recursive = true)
```

4.8. rmdir

```
def rmdir(path: str, recursive: bool = False) -> maix.err.Err
```

Remove a directory

item	description
param	**path**: path to remove **recursive**: if true, remove directory recursively, otherwise, only remove empty directory, default is false
return	err::ERR_NONE(err.Err.ERR_NONE in MaixPy) if success, other error code if failed

C++ defination code:

```
err::Err rmdir(const std::string &path, bool recursive = false)
```

4.9. remove

```
def remove(path: str) -> maix.err.Err
```

Remove a file

item	description
param	**path**: path to remove
return	err::ERR_NONE(err.Err.ERR_NONE in MaixPy) if success, other error code if failed

> C++ defination code:
>
> ```
> err::Err remove(const std::string &path)
> ```

4.10. rename

```
def rename(src: str, dst: str) -> maix.err.Err
```

Rename a file or directory

item	description
param	**src**: source path **dst**: destination path, if destination dirs not exist, will auto create
return	err::ERR_NONE(err.Err.ERR_NONE in MaixPy) if success, other error code if failed

C++ defination code:

```
err::Err rename(const std::string &src, const std::string &dst)
```

4.11. sync

```
def sync() -> None
```

Sync files, ensure they're wrriten to disk from RAM

C++ defination code:

```
void sync()
```

4.12. getsize

```
def getsize(path: str) -> int
```

Get file size

item	description
param	**path**: path to get size
return	file size if success, -err::Err code if failed

C++ defination code:

```
int getsize(const std::string &path)
```

4.13. dirname

```
def dirname(path: str) -> str
```

Get directory name of path

item	description
param	**path**: path to get dirname
return	dirname if success, empty string if failed

> C++ defination code:
>
> ```
> std::string dirname(const std::string &path)
> ```

4.14. basename

```
def basename(path: str) -> str
```

Get base name of path

item	description
param	**path**: path to get basename
return	basename if success, empty string if failed

> C++ defination code:
>
> ```
> std::string basename(const std::string &path)
> ```

4.15. abspath

```
def abspath(path: str) -> str
```

Get absolute path

item	description
param	**path**: path to get absolute path
return	absolute path if success, empty string if failed

> C++ defination code:
>
> ```
> std::string abspath(const
> std::string &path)
> ```

4.16. getcwd

```
def getcwd() -> str
```

Get current working directory

item	description
return	current working directory absolute path

> C++ defination code:
>
> ```
> std::string getcwd()
> ```

4.17. realpath

```
def realpath(path: str) -> str
```

Get realpath of path

item	description
param	**path**: path to get realpath
return	realpath if success, empty string if failed

> C++ defination code:
>
> ```
> std::string realpath(const std::string &path)
> ```

4.18. splitext

```
def splitext(path: str) -> list[str]
```

Get file extension

item	description
param	**path**: path to get extension
return	prefix_path and extension list if success, empty string if failed

> C++ defination code:
>
> ```
> std::vector<std::string> splitext(const std::string &path)
> ```

4.19. listdir

```
def listdir(path: str, recursive: bool = False, full_path: bool = False) -> list[str]
```

List files in directory

item	description
param	**path**: path to list **recursive**: if true, list recursively, otherwise, only list current directory, default is false **full_path**: if true, return full path, otherwise, only return basename, default is false
return	files list if success, nullptr if failed, you should manually delete it in C++.

> C++ defination code:
>
> ```
> std::vector<std::string> *listdir(const std::string &path, bool recursive = false, bool full_path = false)
> ```

4.20. open

```
def open(path: str, mode: str) -> File
```

Open a file, and return a File object

item	description
param	**path**: path to open **mode**: open mode, support "r", "w", "a", "r+", "w+", "a+", "rb", "wb", "ab", "rb+", "wb+", "ab+"
return	File object if success(need to delete object manually in C/C++), nullptr if failed

> C++ defination code:
>
> ```
> fs::File *open(const std::string &path, const std::string &mode)
> ```

4.21. tempdir

```
def tempdir() -> str
```

Get temp files directory

item	description
return	temp files directory

> C++ defination code:
>
> ```
> std::string tempdir()
> ```

5. Class

5.1. File

File read write ops

> C++ defination code:
>
> ```
> class File
> ```

5.1.1. __init__

```
def __init__(self) -> None
```

Construct File object

item	description
type	func
static	False

> C++ defination code:
>
> ```
> File()
> ```

5.1.2. open

```
def open(self, path: str, mode: str)
-> maix.err.Err
```

Open a file

item	description
type	func
param	**path**: path to open **mode**: open mode, support "r", "w",

item	description
	"a", "r+", "w+", "a+", "rb", "wb", "ab", "rb+", "wb+", "ab+"
return	err::ERR_NONE(err.Err.ERR_NONE in MaixPy) if success, other error code if failed
static	False

C++ defination code:

```
err::Err open(const std::string &path, const std::string &mode)
```

5.1.3. close

```
def close(self) -> None
```

Close a file

item	description
type	func
static	False

C++ defination code:

```
void close()
```

5.1.4. read

```
def read(self, size: int) -> list[int]
```

Read data from file API2

item	description
type	func
param	**size**: max read size
return	bytes data if success(need delete manually in C/C++), nullptr if failed
static	False

> C++ defination code:
>
> ```
> std::vector<uint8_t> *read(int size)
> ```

5.1.5. readline

```
def readline(self) -> str
```

Read line from file

item	description
type	func
return	line if success, None(nullptr in C++) if failed. You need to delete the returned object manually in C/C++.
static	False

C++ defination code:

```
std::string *readline()
```

5.1.6. eof

```
def eof(self) -> int
```

End of file or not

item	description
type	func
return	0 if not reach end of file, else eof.
static	False

C++ defination code:

```
int eof()
```

5.1.7. write

```
def write(self, buf: list[int]) -> int
```

Write data to file API2

item	description
type	func
param	**buf**: buffer to write

item	description
return	write size if success, -err::Err code if failed
static	False

C++ defination code:

```
int write(const
std::vector<uint8_t> &buf)
```

5.1.8. seek

```
def seek(self, offset: int, whence:
int) -> int
```

Seek file position

item	description
type	func
param	**offset**: offset to seek **whence**: @see maix.fs.SEEK
return	new position if success, -err::Err code if failed
static	False

C++ defination code:

```
int seek(int offset, int whence)
```

5.1.9. tell

```
def tell(self) -> int
```

Get file position

item	description
type	func
return	file position if success, -err::Err code if failed
static	False

> C++ defination code:
>
> ```
> int tell()
> ```

5.1.10. flush

```
def flush(self) -> maix.err.Err
```

Flush file

item	description
type	func
return	err::ERR_NONE(err.Err.ERR_NONE in MaixPy) if success, other error code if failed
static	False

> C++ defination code:
>
> ```
> err::Err flush()
> ```